Day 5 - OOPs Association, Aggregation & Composition

ITSRUNTYM

Object-Oriented Relationships Overview

Why Study Object Relationships?
When designing systems (like banking apps, e-commerce, or social networks), understanding **how objects relate** to each other helps to:

- Model real-world scenarios
- Define **ownership** and **lifecycles**
- Prevent memory leaks or bugs
- Apply principles like **SOLID**, **DRY**, **composition over inheritance**

1. Association – "Has-a" Relationship

Definition:
Association is a **general relationship** where one class **uses or interacts with another**, without ownership.

- It's **bidirectional** or **unidirectional**
- Objects are **independent**
- No part-of relationship

Types of Association:

Type	Meaning
One-to-One	A person has one passport
One-to-Many	A teacher has many students
Many-to-One	Many students share one college
Many-to-Many	Many students enroll in many courses

1. One-to-One Association

Definition:
One object of Class A is associated with **exactly one** object of Class B and vice versa.

Real-life Example:

- A **Person** has one **Passport**.
- A **Country** has one **President**.

Each object maps to **only one** counterpart.

Java Code Example:

```
class Passport {
    String passportNumber;
}
class Person {
    String name;
    Passport passport; // One-to-One
}
```

Here, each Person is linked to only one Passport.

UML Notation:

```
Person ——— 1 -------- 1 ———> Passport
```

Use-case:

- When two entities are **exclusive and tightly linked**, but **independent** in lifecycle.

2. One-to-Many Association

Definition:

One object of Class A is associated with **multiple objects** of Class B.

Real-life Example:

- A **Teacher** teaches many **Students**.
- A **Department** has many **Employees**.

One "parent", many "children".

Java Code Example:

```
class Student {
    String name;
}
```

```
class Teacher {
    String name;
    List<Student> students = new ArrayList<>(); // One-to-Many
}
```

UML Notation:

```
Teacher ——— 1 -------- * ———> Student
```

Use-case:

- Parent-child relationship where **child objects are logically grouped** under a parent but can **exist independently**.

3. Many-to-One Association

Definition:

Multiple objects of Class A are associated with **a single object** of Class B. This is the **reverse** of one-to-many.

Real-life Example:

- Many **Students** study at the same **College**.
- Many **Employees** work under one **Manager**.

Java Code Example:

```
class College {
    String name;
}
class Student {
    String name;
    College college; // Many-to-One
}
```

UML Notation:

```
Student ——— * -------- 1 ———> College
```

Use-case:

- **Shared resource** or common grouping (e.g., department, project, or company).
- Students can exist independently but are associated with a single college.

4. Many-to-Many Association

Definition:

Multiple objects of Class A can be associated with **multiple objects** of Class B.

Real-life Example:

- Many **Students** enroll in many **Courses**.
- Many **Authors** write many **Books**.

This is a **bidirectional association**.

Java Code Example:

```
class Course {
    String name;
    List<Student> students = new ArrayList<>();
}
class Student {
    String name;
    List<Course> courses = new ArrayList<>();
}
```

Here:

- Each Student can take multiple Courses
- Each Course can have multiple Students

UML Notation:

```
Student ——— * -------- * ———> Course
```

Use-case:

- Complex relationships with **cross-linking**.
- Requires intermediate mapping or **join table** in DB (many-to-many is **not directly supported** in most relational DBs).

Summary Table

Type	Cardinality	Real-world Example	Java Structure
One-to-One	1 : 1	Person ↔ Passport	Object field
One-to-Many	1 : *	Teacher ↔ Students	List/Set field
Many-to-One	* : 1	Students ↔ College	Single field reference

Many-to-Many	* : *	Students ↔ Courses	List in both classes

Code Example:

```
class Student {
   String name;
}
class Teacher {
   List<Student> students = new ArrayList<>(); // Association
}
```

Real-Life Example:

- A **Doctor** and **Patient**:
  - The doctor treats many patients
  - A patient can visit multiple doctors
  - **No ownership**, they exist independently

Characteristics:

Feature	Value
Ownership	No
Lifecycle link	Independent
Coupling	Loose
Reusability	High

When to Use:

- Temporary links
- Logically related but independent entities
- E.g., Messaging system (Sender ↔ Receiver)

UML Notation:

```
Customer ---------> Order
```

- Solid line with arrow

2. Aggregation – "Has-a" With Weak Ownership

Definition:
The **whole (container)** object uses or contains the **part (contained)** object, but **does not own it**.

Aggregation is a **special form of association**. It represents a **whole-part** relationship where:

- The **"whole" uses the part**, but doesn't own it.
- The part **can exist independently** of the whole.

Why is it Weak Ownership?
"Weak ownership" means:

- The parent **references** the child, but **does not control its lifecycle**.
- The child **was created externally**, or can be **shared**.
- The child **can exist without** the parent.

Real-World Example:
University and Department:

- A **University** "has-a" list of **Departments**.
- But a **Department** can:
  - exist **without** the University.
  - be **moved** to another University.

Hence, the **University doesn't own** the Department strongly — this is **aggregation**.

Java Code Example:

```
class Department {
   String name;
}
class University {
   String name;
   List<Department> departments;
University(String name, List<Department> departments) {
      this.name = name;
      this.departments = departments; // Aggregation
   }
}
```

Key Point:

- The Department objects were **created outside** and **passed into** the University.
- So, the University **does not own** them — only **aggregates** them.

Lifecycle Explanation:

- If we destroy the University object (set it to null), the Department objects **still exist** in memory.
- That's **weak ownership** — no dependency on the parent's lifecycle.

Ownership Example Diagram:

University ◇———> Department
(Aggregation)

- **Hollow Diamond (◇)** in UML indicates Aggregation.

Characteristics of Aggregation

Feature	Value
Relationship	Whole–Part
Ownership	Weak
Part's lifecycle	Independent from whole
Object creation	Part is created externally
Object reuse	Part can be shared or reused
Memory Management	GC doesn't destroy part with whole
UML Notation	Hollow Diamond (◇)

When to Use Aggregation?

Use **Aggregation** when:

1. The parent **uses** the child but does **not control it**.
2. The child object can **exist independently**.
3. The child may be **shared** across different parents.
4. You want **modular, loosely coupled** design.

Real-life Aggregation Examples

Whole	Part	Explanation
Car	Engine	Engine can be used in different cars

Library	Book	Books are added to multiple libraries
School	Teacher	A teacher may teach in multiple schools
Department	Employee	Employees can transfer or move
Company	Projects	Projects may continue even after company closure

Code Example:

```
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}
class Car {
    Engine engine; // Aggregation
Car(Engine engine) {
        this.engine = engine;
    }
}
```

- **Engine is passed externally**
- If **Car** is destroyed, **Engine** still exists

Characteristics:

Feature	Value
Ownership	Weak
Lifecycle link	Optional
Reusability	High
Coupling	Medium

UML Notation:

Car ◇———> Engine

- Hollow diamond indicates aggregation

When to Use:
- Reusable components (like Engine, DatabaseConnection)
- Objects shared across multiple parents
- Shared ownership (a book used in different libraries)

3. Composition – "Part-of" With Strong Ownership

Definition:
Composition is a **strong form of aggregation** where the **"whole" completely owns the part**.
- The child object **cannot exist without** the parent.
- The child's **lifecycle is tightly bound** to the parent.
- The parent is **responsible for creating and destroying** the child.
- It's a **strong form of Aggregation**.

If the parent object is destroyed, the child object must also be destroyed.
This makes the ownership **strong** and **exclusive**.

Why It's Called Strong Ownership?
Because:
1. The **child cannot exist independently**.
2. The **parent fully manages the child's lifecycle**.
3. The child is often **created inside the parent**.
4. The child object is **private** to the parent.
5. The child is not shared.

This tight control makes the ownership **strong**.

Real-World Analogies

Whole (Parent)	Part (Child)	Can part exist independently?	Composition?
Human	Heart	No	Yes
House	Room	No	Yes
Book	Chapter	No	Yes

Car	Steering System	No	Yes
Order	Order Items	No	Yes

Java Code Example – Composition

Human and Heart Example:

```
class Heart {
    void beat() {
        System.out.println("Heart is beating...");
    }
}
class Human {
    private final Heart heart;
public Human() {
        this.heart = new Heart(); // Composition → created internally
    }
void live() {
        heart.beat();
    }
}
```

Key Points:

- Heart is **created inside Human** → tight binding.
- **No reuse** of Heart.
- Heart **cannot exist** without Human.

Lifecycle Control:

```
Human h = new Human(); // creates heart internally
h = null;              // Human object destroyed
// → Heart object also lost (eligible for GC)
```

UML Representation – Composition

Human ◆————> Heart

- Filled diamond (◆) indicates **composition**.
- The arrow points from **whole → part**.

Characteristics of Composition

Feature	Value
Relationship type	Strong “part-of”
Ownership	Strong
Lifecycle link	Tied to parent
Object creation	Inside parent
Coupling	High (tight)
Object reuse	Not reusable
Encapsulation	High (private members)
UML notation	Filled Diamond

When to Use Composition

Use Composition when:

- The child **makes no sense on its own**.
- The child is a **part of the internal structure**.
- You want **strong encapsulation**.
- You want **automatic cleanup** with the parent.

Aggregation vs Composition

Feature	Aggregation	Composition
Ownership	Weak (shared)	Strong (exclusive)
Lifecycle	Independent	Tightly bound
Object creation	External	Internal
Object sharing	Yes	No
Encapsulation	Partial	Complete
UML	Hollow Diamond (◇)	Filled Diamond (◆)
Example	Car ↔ Engine	House ↔ Room

Composition in Real-World Applications

Examples:

App Type	Composition Example
E-Commerce	Order → OrderItems
Document Editor	Document → Pages
HR System	Employee → Address
Game Engine	Game → GameObjects

Social Media	Post → Comments

Order System Example

```
class OrderItem {
    String itemName;
}
class Order {
    private List<OrderItem> items = new ArrayList<>();
public void addItem(String itemName) {
        items.add(new OrderItem(itemName)); // composition
    }
}
```

- If the Order is deleted, OrderItems are also deleted.
- **Items are not shared** between orders → tight ownership.

Detailed Comparison Table

Feature	Association	Aggregation	Composition
Definition	Usage relation	Whole-Part relation	Exclusive Part-of
Ownership	No	Weak	Strong
Lifecycle dependency	No	No	Yes
Coupling	Loose	Moderate	Tight
Reusability	High	Moderate	None
UML Notation	Line	Hollow Diamond ◇	Filled Diamond ◆
Java Example	Teacher–Student	Car–Engine	Human–Heart

Real World System Modelling Examples

Social Network:

Relationship	Type	Reason
User → Profile	Composition	Profile is part of User
Post ↔ Comments	Composition	Comments live only with post
User ↔ Friends	Association	Friends are separate users
Post ↔ Likes	Aggregation	Likes can exist across posts

E-Commerce:

Relationship	Type	Reason
Order → OrderItems	Composition	Items destroyed with order
User ↔ Cart	Aggregation	Cart shared or persisted
Product ↔ Category	Aggregation	Product can belong to multiple
Admin ↔ Reports	Association	Temporary usage

Let's compare:

Type	Ownership Level	Lifecycle
Association	None	Separate
Aggregation	Weak	Independent
Composition	Strong	Bound to parent

1. Association

Parent ↔ Child: Loose Link

- No ownership in either direction.
- Both objects are **independent**.

Parent created/destroyed → No effect on child
Child created/destroyed → **No effect** on parent

Example:

```
class Student {}
class Library {}
Student s = new Student();
Library l = new Library();
l.allowEntry(s); // temp relationship
```

- Destroying Library doesn't affect Student.
- Destroying Student doesn't affect Library.

Both exist separately
Association = just a **reference**, not a dependency

2. Aggregation

Parent → Child: Uses but doesn't own
- The child is passed in from outside.
- The parent **keeps a reference** to the child.
- Child may be **shared** across other parents.

Parent destroyed → Child still exists
Child destroyed → **Parent must handle it (null checks, etc.)**

Example:

```
Engine e = new Engine();      // created independently
Car c1 = new Car(e);          // car uses engine (aggregation)
Car c2 = new Car(e);          // same engine used again
```

- If Car c1 is destroyed → Engine e still exists
- If Engine e is destroyed → both c1 and c2 fail (but they didn't own it)

Parent and child can exist independently,
but **parent may break** if it tries to access a destroyed child it does not own.

3. Composition

Parent owns the child.

- The parent is responsible for creating and destroying the child.
- The child cannot exist independently.

Parent destroyed → Child is destroyed
Child destroyed independently → Not possible (child is private/internal)

Example:

```
class Car {
   private Engine engine = new Engine(); // created inside → composition
}
```

- If Car is destroyed → Engine is gone (eligible for GC)
- If you try to get the Engine outside — not allowed → strong encapsulation

What About Child → Parent?

- Child has **no separate identity**.
- It **lives and dies with the parent**.
- Child cannot exist **before** the parent.
- It's created **inside** the parent — so **no reverse effect**.

Final Comparison: Lifecycle Impact (Both Directions)

Relationship	Parent Destroyed → Child?	Child Destroyed → Parent?	Can Child Exist Without Parent?
Association	No effect	No effect	Yes
Aggregation	No effect	May break if accessed	Yes
Composition	Child is destroyed too	Not allowed (private/internal)	No

Visual Diagram

```
Association:     [Parent] <---> [Child]    // separate lifecycles
Aggregation:     [Parent] ---> [Child]     // weak reference
Composition:     [Parent] owns [Child]     // tight ownership, bound lifecycle
```

Summary

- In **association**, parent and child are **completely independent**
- In **aggregation**, parent **uses** the child but **does not control** it
- In **composition**, parent **creates**, **owns**, and **destroys** the child — so the child is tightly **bound to the parent's lifecycle**